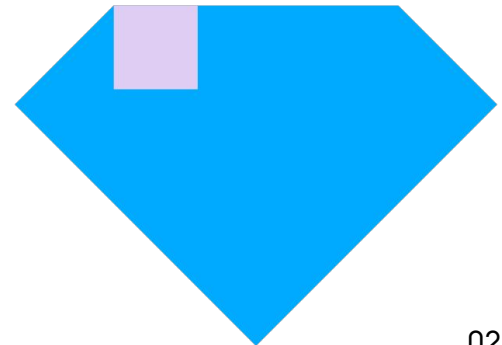




Это полностью
прикладной курс.



Шаповалов Александр



«Вижу,
как медленно
внедряется
AI в бигтехе»

Продвинутый LLM промптинг

02

Сегодня узнаете

01

Еще немного про AI
в мире

02

Обзор промт техник

03

RTCF

04

Chain of X

05

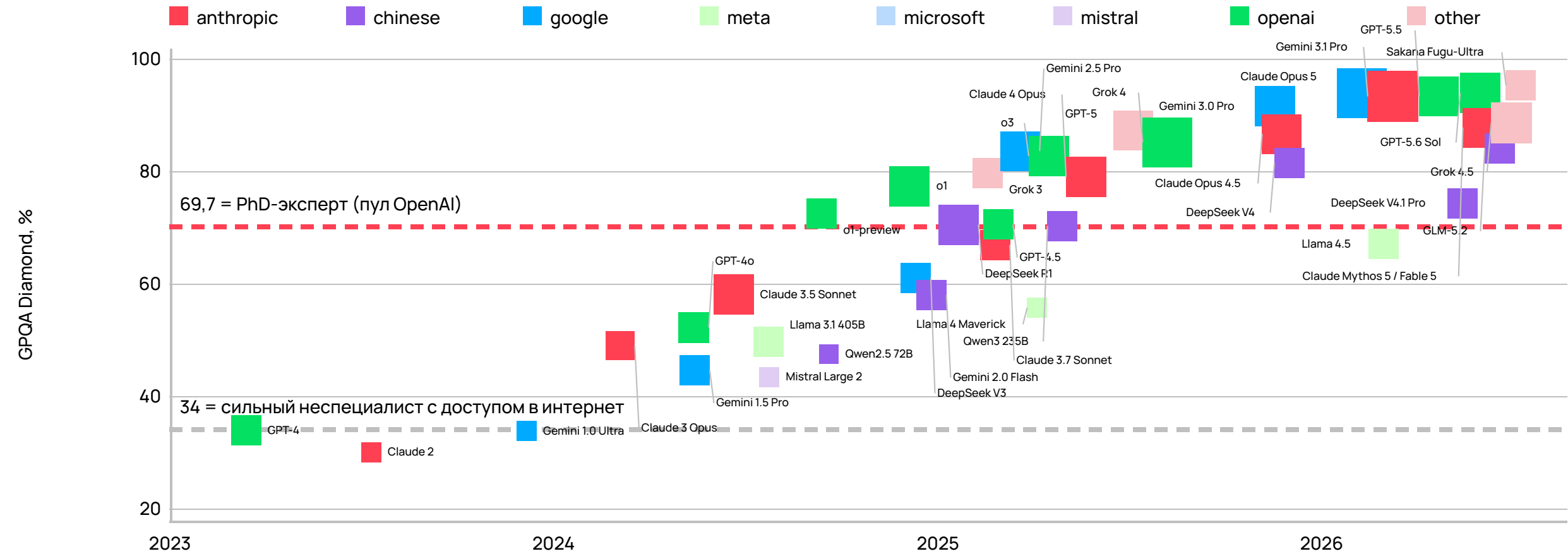
От ассистентов
к агентам

Р1. Еще немного про AI

1. Экспертный уровень моделей
2. Планы Alibaba на 2026 г
3. Возможности LLM на реальных задачах

Языковые модели приближаются к экспертному уровню

Экспертность моделей: GPQA Diamond vs человек-эксперт (2023–2026)

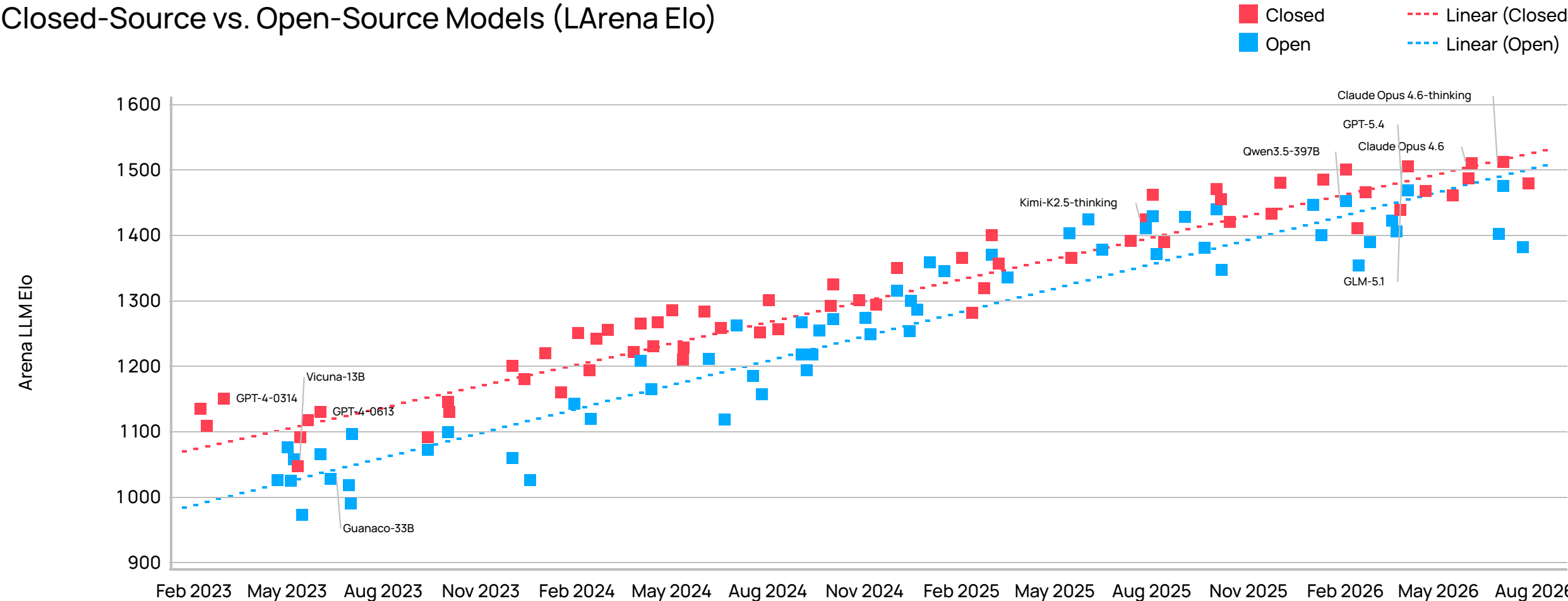


Источники: Artificial Analysis, Epoch AI, отчёты вендоров (OpenAI, Google, Anthropic и др.), сводные лидерборды по состоянию на август 2026.

Оценки по независимым методикам могут отличаться от заявленных вендором на несколько п. п. Размер точки — условный индикатор значимости релиза, не строгая метрика.

Closed-Source vs. Open-Source Models

Closed-Source vs. Open-Source Models (LArena Elo)



Источники: Arena Leaderboard Dataset (Imarena-ai / Arena Intelligence, HuggingFace), сводка BenchLM.ai (обновлено 2026-07-27).
Показаны лидирующая модель каждого месяца по категории + иллюстративный разброс остальных моделей вокруг тренда; отдельные точки не привязаны к конкретным релизам.

Alibaba обнародовали дорожную карту по Qwen в 2026



Длина контекста: 1M → 100M токенов (10 млн. строк кода, 750 книг)



Параметры: 1 трлн. → 10 трлн.



Данные: 10 трлн. → 100 трлн.



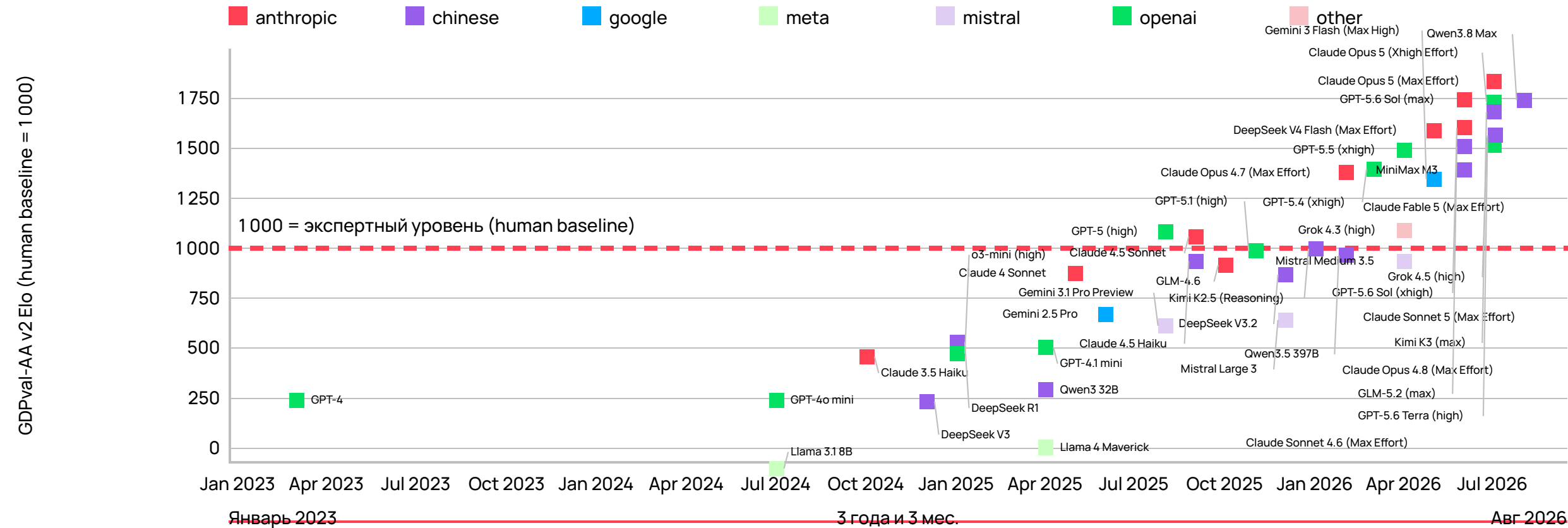
OpenAI: Возможности LLM на реальных задачах цифровых профессий

Как оценивали LLM на реальной работе

- ✓ Поле задач: 9 крупнейших отраслей по вкладу в ВВП → 44 «цифровые» профессии.
- ✓ Датасет задач: 1 320 реальных заданий. Каждое — «запрос + реф-файлы → рабочий продукт» (таблицы, слайды, PDF, изображения, видео).
- ✓ Эксперты: практики (ср. стаж 14 лет) создавали И оценивали задачи; строгий отбор (интервью, бэкграунд-чек, тренинг).
- ✓ Контроль качества: авто-скрининг моделью + минимум 3 (в среднем 5) раундов ревью.
- ✓ Как мерили качество: слепые попарные сравнения «модель vs человек-эксперт» ⇒ win-rate (учитывает корректность, структуру, стиль и формат).
- ✓ Скорость/стоимость: оценивали скорость и стоимость выполнения задач человеком и моделью.

Прогресс качества моделей x3 за 1.5 года

GDPval: производительность моделей на реальных задачах (2023–2026)

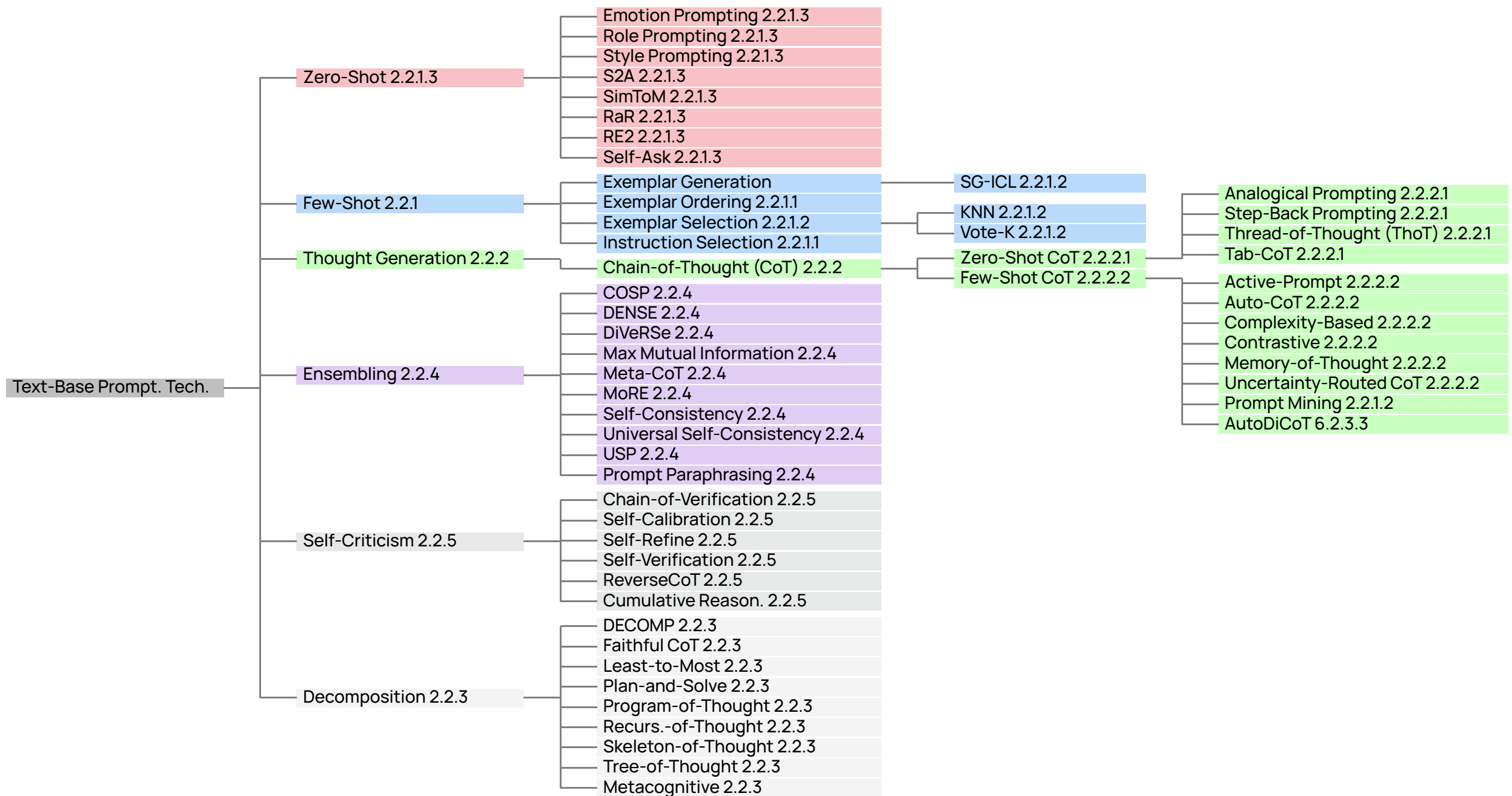


Источники: OpenAI GDPval (arXiv 2510.04374); GDPval-AA v2 — независимая переоценка Artificial Analysis (220 задач из набора GDPval, Elo, судья — LLM, по состоянию на 10.08.2026). Elo заякорен так, что человек-эксперт = 1 000.

Часть моделей показана в разных режимах reasoning-effort (Low/Medium/High/Xhigh/Max) — это разные точки на графике для одной и той же модели.

Р2. Обзор промт техник

1. Список метрик и частотность использования
2. Сравнение техник по их скорости и точности



Citation Counts of Prompting Techniques

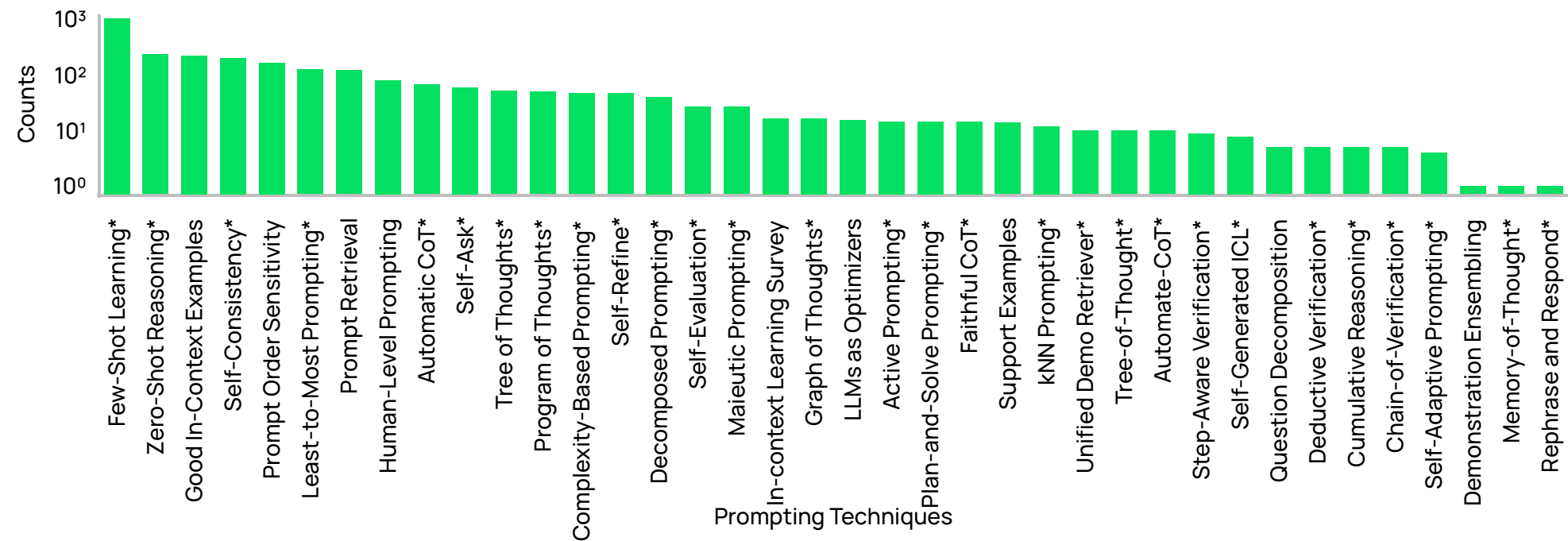
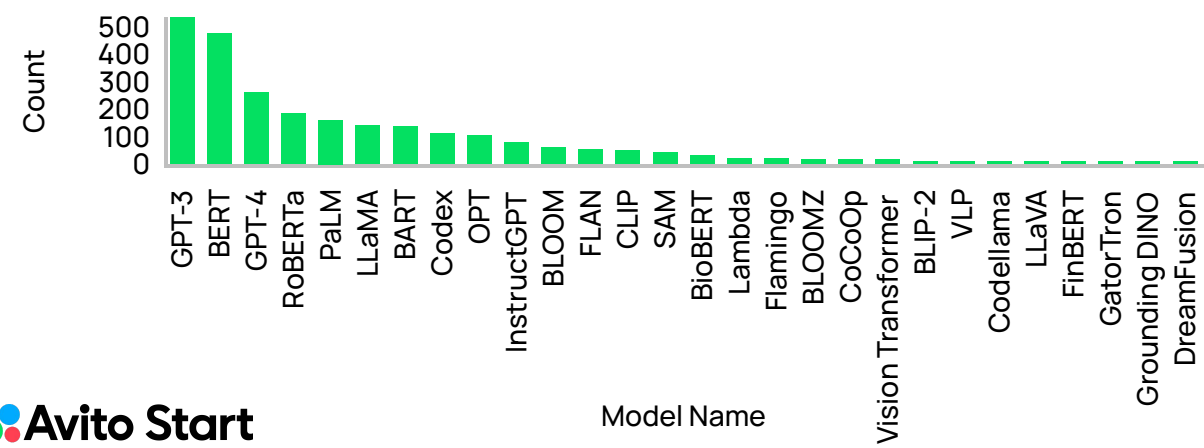


Figure 2.11: Citation Counts of Prompting Techniques. The top 25 papers in our dataset, measured by how often they are cited by other papers in our dataset. Most papers here are prompting techniques*, and the remaining papers contains prompting advice.

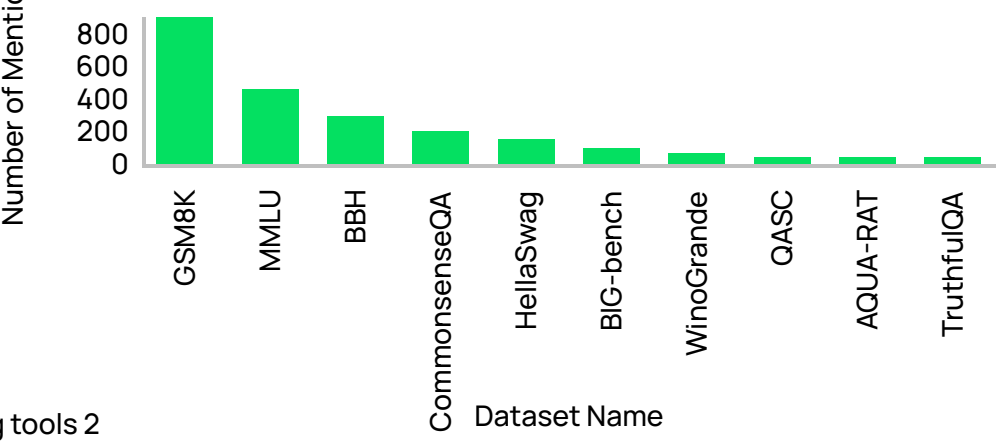
Counts of Model Mentions in Dataset

Figure 2.9: Citation Counts of GenAI Models



Dataset Mentions in Papers

Figure 2.10: Citation Counts of Datasets



Comparative Study and Analysis of Prompting Techniques using Gemini API Model and Reasoning Frameworks



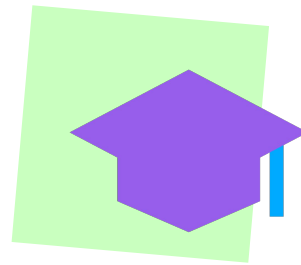
Rekha V

Associate Professor
Department of CSE, SoET
Christ University
Email: rekha.v@christuniversity.in



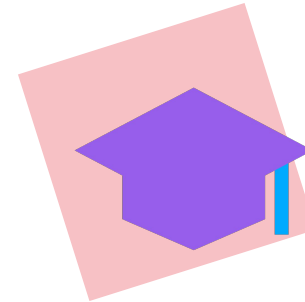
Prakash G L

Associate Professor
Department of Information Science
and Engineering, BMSIT&M, VTU
Email: glprakash@bmsit.in



Boppuru Rudra Prathap

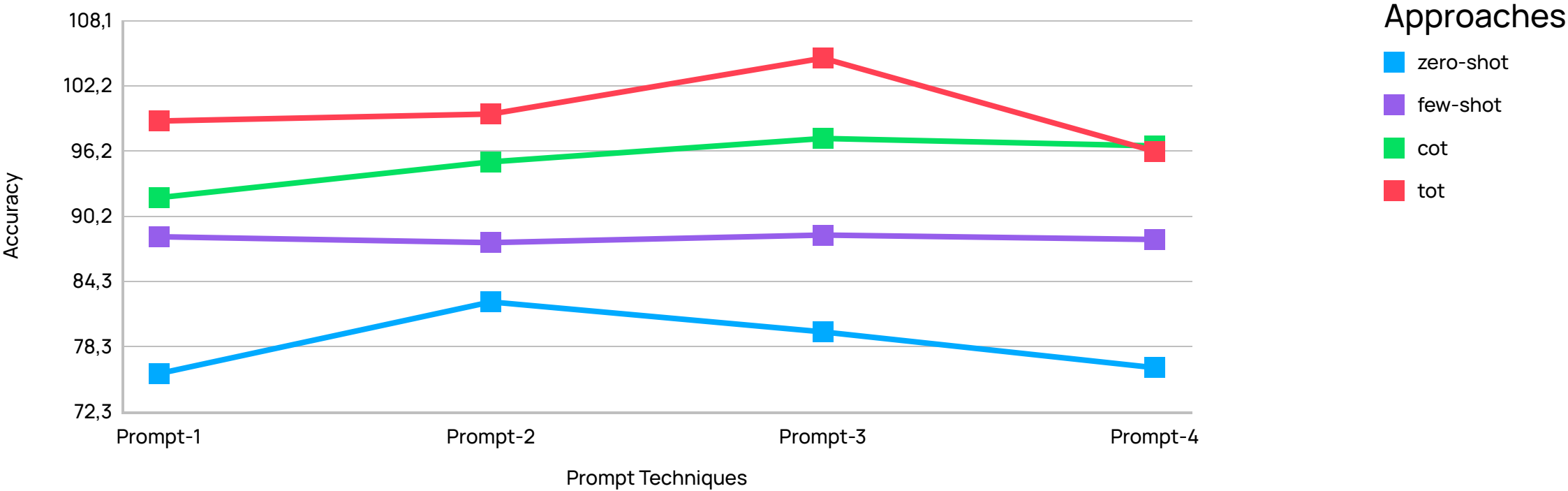
Associate Professor
Department of CSE, SoET
Christ University
Email:
boppuru.prathap@christuniversity.in



Vandana Reddy

Assistant Professor
Department of CSE, SoET
Christ University
Email:
vandana.reddy@christuniversity.in

Comparison of Accuracy across Approaches (gemini-large)

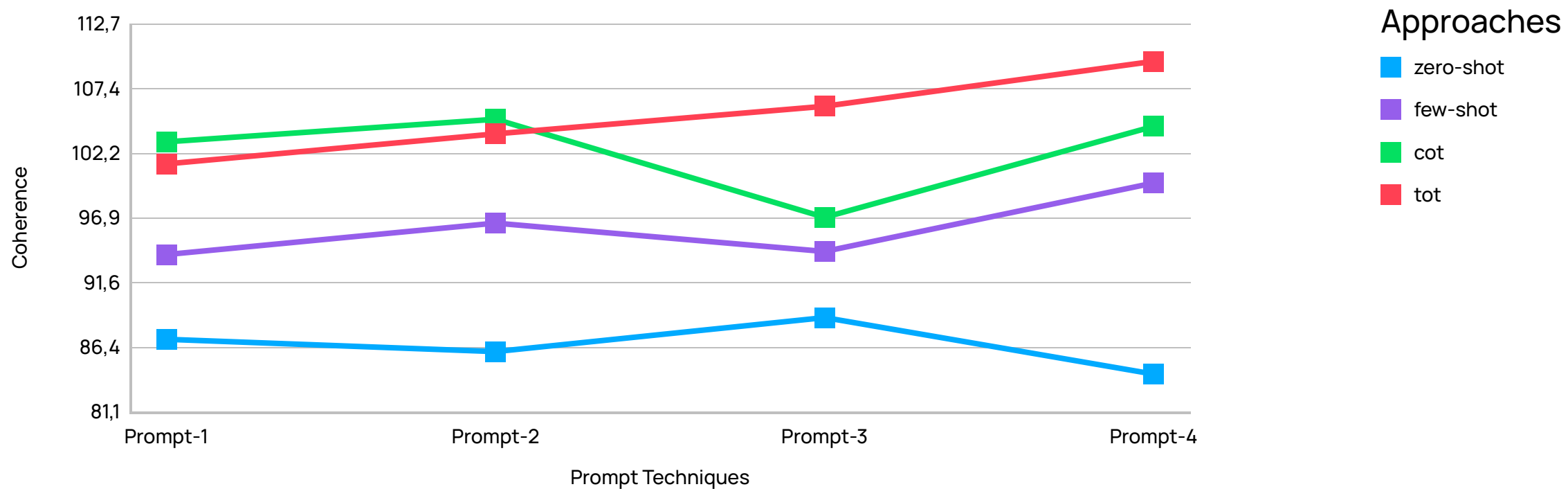


Comparison of Accuracy of Gemini-Large model with different prompts

CoT	Few-Shot	ToT	Zero-Shot
91,8883	88,3128	98,9039	75,7839
95,1659	87,7736	99,5428	82,3439
97,3098	88,4659	104,6570	79,6005
96,6281	88,0595	96,1262	76,3234

AI engineering tools 2

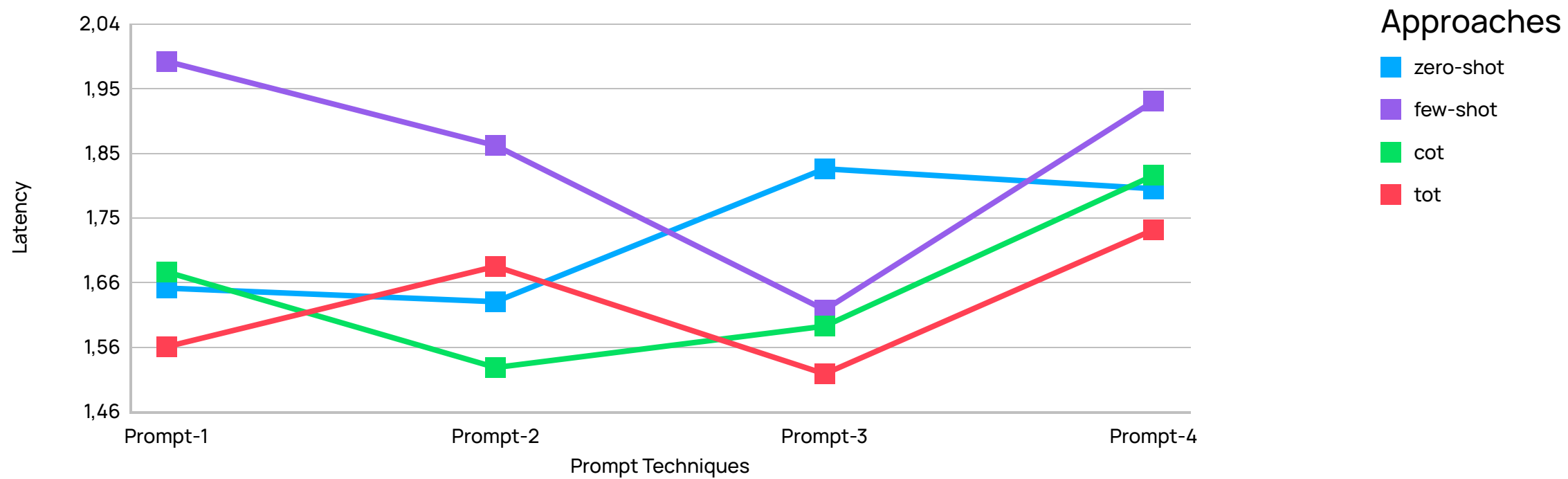
Comparison of Coherence across Approaches (gemini-large)



Comparison of Coherence of Gemini-Large model with different prompts

CoT	Few-Shot	ToT	Zero-Shot
103,0700	93,9017	101,2740	87,0100
104,9090	96,4593	103,7200	86,0104
96,9263	94,1536	105,9630	88,7759
104,3270	99,7204	109,6180	84,1781

Comparison of Latency across Approaches (gemini-large)



Comparison of Latency of Gemini-Large model with different prompts

CoT	Few-Shot	ToT	Zero-Shot
1,67087	1,98779	1,55817	1,64652
1,52699	1,86091	1,67918	1,62607
1,58925	1,61383	1,51745	1,82605
1,81604	1,92763	1,73436	1,79607

P3. RTCF

1. Промт техника на все случаи жизни
2. Смотрим на примере

R — Role

Роль / Персона

Что это

Кем должна быть модель?

Зачем

Задаёт тон, стиль мышления и доступ к специализированным знаниям.

C — Context

Контекст / Данные

Что это

Вводные данные, ограничения, история, «дано».

Зачем

Без контекста модель Галлюцинирует (Zero-shot).
С контекстом — решает конкретную задачу.

T – Task

Задача / Действие

Что это

Глагол. Что конкретно
нужно сделать?

Зачем

Четкая инструкция
к действию.

F — Format

Формат вывода

Что это

В каком виде вернуть ответ? (JSON, Markdown, Таблица, Python list).

Зачем

Чтобы результат можно было использовать дальше (парсить кодом).

The Perfect ChatGPT Prompt Formula

By Robin Delta

Mandatory

Task

+

Context

Important

Exemplar

+

Persona

Nice to have

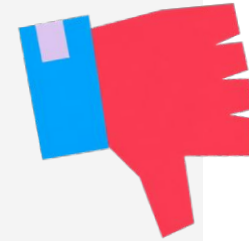
Format

+

Tone

Пример неэффективного промта

Неэффективный промпт
(Zero-Context):



«Посмотри это
и скажи, что не так»

Пример хорошего промта

RCTF Промт (Structured):

(R)ole: Ты эксперт по безопасности Python (SecOps Engineer).

(C)ontext: У нас есть Legacy-код API, написанный 5 лет назад. Мы готовимся к аудиту. Вот фрагмент кода: [CODE].

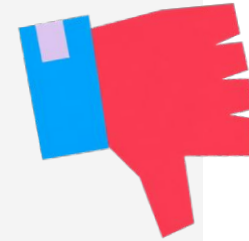
(T)ask: Проведи аудит на уязвимости (SQL Injection, XSS). Не пиши исправленный код, только перечисли найденные дыры.

(F)ormat: Верни результат строго в JSON формате: [{"line": int, "vulnerability": str, "severity": "high/med"}].



Тестируем RTCF

Промт 1. Без RCTF



«Придумай мне
несколько
пользователей для
тестирования
магазина. Имя, почта
и что они купили.
Сделай штук 5–6.»

Тестируем RTCF

Промт 2. C RCTF



Role: Ты Senior QA Automation Engineer. Ты специализируешься на тестировании безопасности и устойчивости бэкенда.

Context: Мы тестируем API оформления заказов. Нам нужны не «хорошие» пользователи, а данные, которые могут сломать систему (edge cases, инъекции, плохие кодировки).

Task: Сгенерируй массив из 5 пользователей. Включи следующие кейсы:

Валидный пользователь.

Пользователь с SQL-инъекцией в имени.

Пользователь с эмодзи и иероглифами в адресе.

Пользователь с экстремально длинным email (200+ символов).

Пользователь с отрицательной суммой заказа.

Format: Верни строго валидный JSON (список объектов). Поля: user_id (uuid), full_name, email, delivery_address, cart_total (float). Не пиши никаких объяснений, только код.

Антипатерны

Over-prompting

Когда пишешь слишком много инструкций, и модель путается.

Negative constraints

«Не делай X» работает хуже, чем «Сделай Y». (Лучше сказать «Пиши кратко», чем «Не пиши длинные предложения»).

Тестируем RTCF

Промт в формате JSON

```
{ "роль": "Senior QA Automation Engineer", "контекст":  
"Тестирование безопасности  
и устойчивости бэкенда API оформления заказов",  
"задача": "Сгенерировать массив объектов для  
проверки граничных сценариев",  
"параметры_вывода": {  
"формат": "валидный JSON массив",  
"количество": 5,  
"схема_объекта": {  
"user_id": "uuid",  
"full_name": "string",  
"email": "string",  
"delivery_address": "string",  
"cart_total": "float"  
}},
```

```
"тестовые_кейсы": [  
{ "тип": "позитивный",  
"описание": "Полностью валидные данные" } ],  
"ограничения": [  
"Вернуть только код JSON",  
"Никакого сопроводительного текста",  
"Использовать кириллицу там, где это уместно для  
контекста" ]}]
```

P4. Chain of X

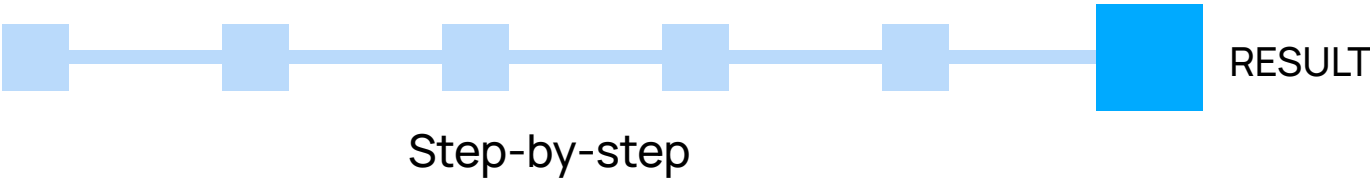
1. Chain of Thoughts
2. Chain of Verification
3. Tree of thoughts

AI Reasoning Methods

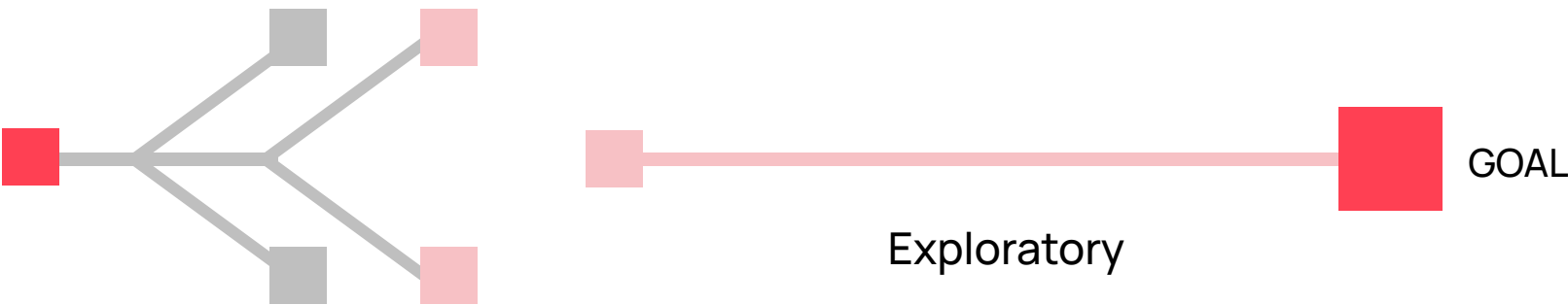
STANDARD



CHAIN OF THOUGHT



TREE OF THOUGHTS



Chain of thoughts

Техника

Мы заставляем модель генерировать промежуточные токены. Чем больше токенов модель пишет в процессе рассуждения, тем больше у неё «времени» (computation time) подумать над финальным ответом.

Ключевая фраза (Magic Spell)

«Let's think step by step» (Давай подумаем шаг за шагом).

Chain of thoughts

Пример: «Ловушка линейного мышления» (Parallelism Trap). Этот пример показывает, как модель не учитывает контекст параллельных процессов.

Обычный промпт

"У меня 5 мокрых рубашек. Чтобы высушить одну рубашку на солнце, требуется 4 часа. Сколько времени потребуется, чтобы высушить все 5 рубашек?"

Ожидаемый ответ: «20 часов». (Модель просто умножает $5 * 4$).

CoT Промпт

"У меня 5 мокрых рубашек. Чтобы высушить одну рубашку на солнце, требуется 4 часа. Сколько времени потребуется, чтобы высушить все 5 рубашек?"

Думай как инженер:

1. Определи, является ли процесс сушки последовательным (одна за другой) или параллельным (все сразу).
2. Есть ли ограничения на ресурсы (веревка вмещает все рубашки?).
3. Посчитай итоговое время."

Ожидаемый ответ: «4 часа. Мы вешаем их одновременно».

Chain of verification

Пример: Избежать галлюцинации. Генерация команды для сложного CLI инструмента (например, FFmpeg), где модели часто путают синтаксис.

Обычный промпт (Standard)

Напиши команду FFmpeg, которая берет видео 'input.mp4', вырезает фрагмент с 00:10 по 00:20, накладывает водяной знак 'logo.png' в правом нижнем углу и сохраняет как GIF в хорошем качестве. Дай только код.

CoV Промпт (Chain of Verification)

Задача: Написать команду FFmpeg для создания GIF из видео с водяным знаком.

Выполни задание в 4 этапа:

1. Draft: Напиши черновик команды.
2. Verification: Сгенерируй 3 вопроса к своему черновику, чтобы проверить его на ошибки (например: «Правильно ли использован фильтр palettegen для качества?», «В каком порядке идут input файлы?», «Правильно ли указаны координаты overlay?»).
3. Check: Ответь на эти вопросы, проверяя синтаксис FFmpeg.
4. Final: Напиши итоговую, исправленную команду.

Tree of thoughts

Проблема

Линейное мышление (CoT) хорошо для задач с одним ответом. Но что, если мы проектируем микросервисную архитектуру? Там нет одного «правильного» пути.

Техника

Мы просим модель сгенерировать 3-4 разных гипотезы, прокритиковать каждую, отсеять слабые и развить сильную.

Tree of thoughts

Пример: Принятие архитектурного решения. Выбор базы данных. Обычный промпт даст банальный совет. ToT даст глубокий анализ.

Обычный промпт (Standard)

Мы делаем стартап — клон Uber. Какую базу данных нам выбрать для хранения геолокации водителей в реальном времени? Postgres или MongoDB?

ToT Промпт (Tree of Thoughts)

Мы делаем клон Uber. Нужно выбрать БД для хранения геопозиции водителей (высокая частота записи, поиск ближайших в радиусе).

Используй метод Tree of Thoughts:

Представь трех экспертов:

- Эксперт А (Фанат PostgreSQL/PostGIS).
- Эксперт Б (Фанат NoSQL/Redis/Mongo).
- Эксперт В (Скептик, System Architect).

Шаг 1: Пусть Эксперт А и Б предложат свои решения с аргументами.

Шаг 2: Пусть Эксперт В жестко раскритикует оба решения (найдет узкие места в производительности записи и масштабировании).

Шаг 3: Пусть эксперты поспорят и придут к компромиссному или гибриднему решению.

Итог: Напиши финальную рекомендацию.

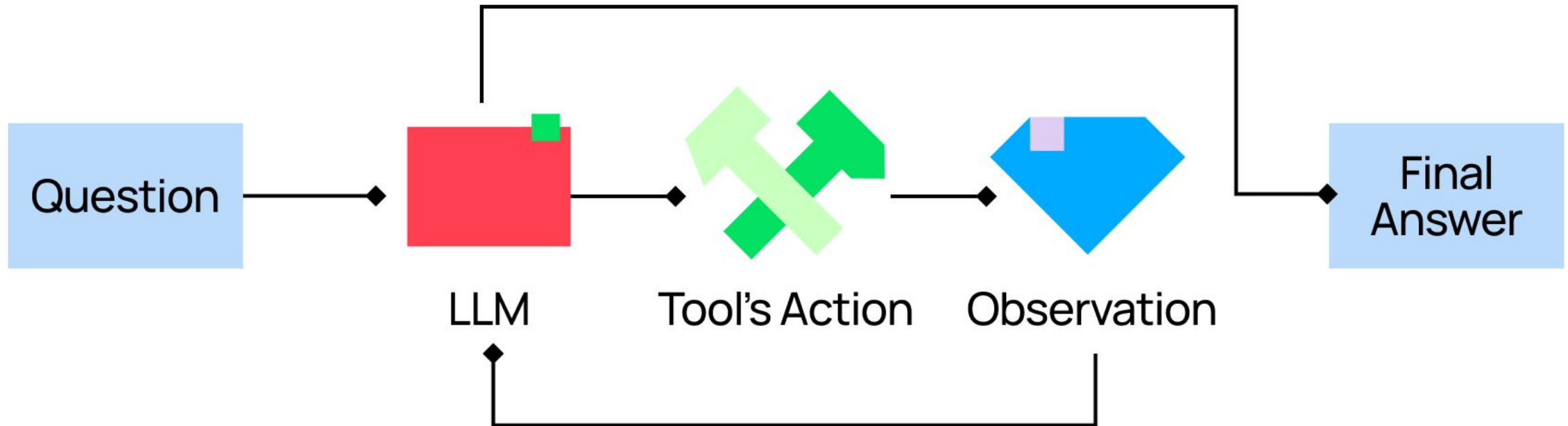
Tree of thoughts

Техника	Аналогия	Для чего (Use Case)	Точность	Latency & Cost
Zero-Shot	Интуиция	Классификация, простой перевод, саммари, форматирование данных.	✳ ✳	Low  (1x токенов)
CoT (Chain of Thought)	Решение в столбик	Математика, логика, написание простых скриптов, дебаг.	✳ ✳ ✳	Medium  (~2-3x токенов на рассуждение)
CoV (Chain of Verification)	Редактура / Фактчекинг	Генерация сложных CLI-команд, использование редких библиотек, юридические/медицинские факты.	✳ ✳ ✳ ✳	High  (Требует 2-3 прогона или длинный контекст)
ToT (Tree of Thoughts)	Совещание экспертов	Архитектура систем, сложные стратегические решения, брейншторм, написание сюжетных линий.	✳ ✳ ✳ ✳ ✳	Very High  (Множество запросов, экспоненциальный рост)

P5. От ассистентов к агентам

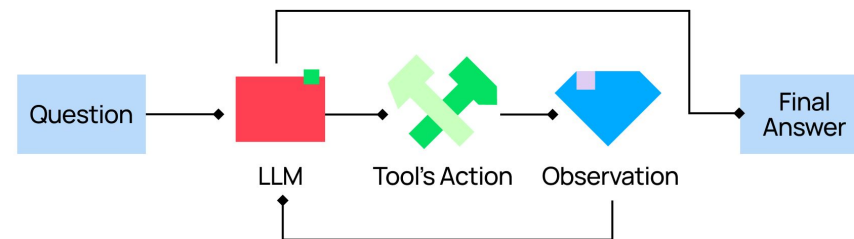
1. ReAct (Reason + Act)
2. RAG (Retrieval Augmented Generation)

ReAct (Reason + Act)



ReAct (Reason + Act)

Компоненты Re-Act-агента:

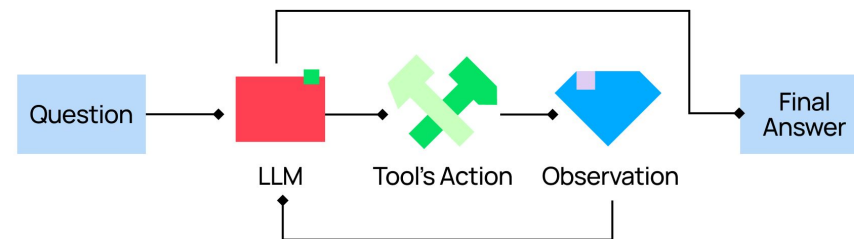


1. Языковая модель (LLM) — «мозг» агента, который генерирует шаги решения. Она выпускает мысли (reasoning traces) и указывает действия для выполнения
2. Инструменты (Tools) — внешние функции или API, которые агент может вызывать для получения информации или выполнения операций. Например, поиск в интернете, вычисления, работа с базой данных и пр. Инструменты регистрируются с именами и описаниями на естественном языке, чтобы модель знала, как ими пользоваться.

3. Исполнитель действий (Agent Executor) — принимает решение модели о действии, вызывает соответствующий инструмент и возвращает результат обратно модели. В контексте Re-Act этот компонент реализует цикл «мышление (Thought) → действие (Act) и наблюдение (Observe)».
4. Состояние агента (State) — накопленная история диалога и действий. Сюда входит запрос пользователя, сообщения модели (мысли/действия) и наблюдения (результаты инструментов). Состояние постоянно обновляется по мере выполнения шагов.

ReAct (Reason + Act)

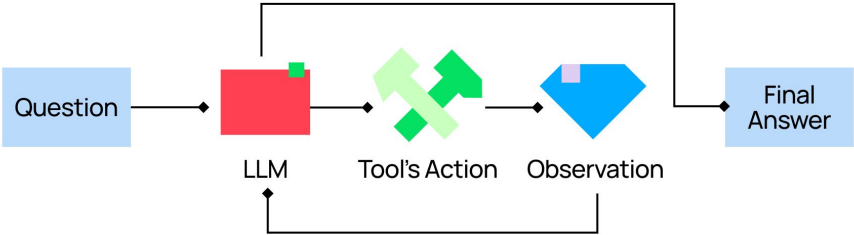
Цикл работы Re-Act агента



1. Получение запроса (Query). Агент принимает входное сообщение пользователя, которое становится исходным состоянием.
2. Генерация рассуждений и действия (Think). Языковая модель на основе текущего состояния генерирует вывод, состоящий из: Thought – внутреннего рассуждения, где агент анализирует задачу и формирует план; Action – указания на выполнение конкретного действия (например, вызов инструмента с определёнными аргументами).
3. Выполнение действия (Action). Если сгенерированный вывод содержит команду (например, Action: Search["query"]), исполнитель вызывает соответствующий инструмент и получает результат.
4. Обновление состояния (Observation). Результат работы инструмента сохраняется как наблюдение и добавляется в историю взаимодействия, после чего цикл повторяется: обновлённое состояние передаётся модели для следующей итерации.
5. Завершение (END). Если модель сгенерировала финальный ответ (например, Answer: ...), цикл прерывается и ответ возвращается пользователю.

ReAct (Reason + Act)

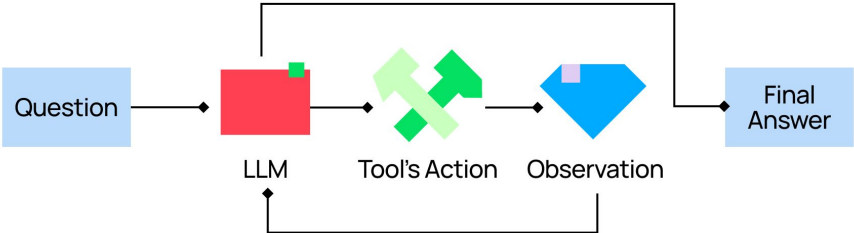
CoT vs Tool-use vs ReAct



	CoT	Tool-use	ReAct
Thought	✓	✗	✓
Action	✗	✓	✓
Observation	✗	✓	✓
Context Extensions	Thought only	Actions + Obs	Thought + Action + Obs

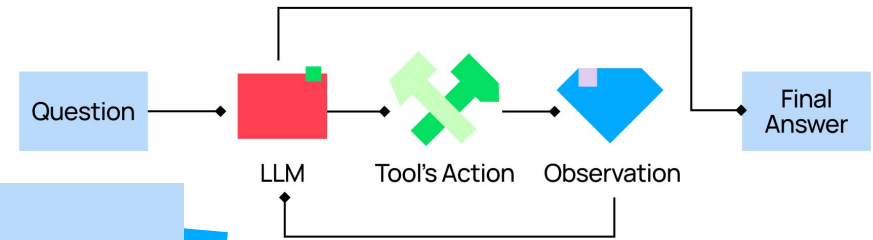
ReAct (Reason + Act)

Table 2: Types of success and failure modes of ReAct and CoT on HotpotQA, as well as their percentages in randomly selected examples studied by human.



	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

ReAct (Reason + Act)



[https://zpnynrtnvatzrjpuexnc.
supabase.co/storage/v1/object/public/uploads/seminar/1756
976310_ReAct____Synergizing
_Reasoning_and_Acting_in_L
anguage_Models.pdf?](https://zpnynrtnvatzrjpuexnc.supabase.co/storage/v1/object/public/uploads/seminar/1756976310_ReAct____Synergizing_Reasoning_and_Acting_in_Language_Models.pdf?)

RAG

Retrieval Augmented Generation

Генерация с дополненной выборкой. Соединяет языковую модель с внешней базой знаний. AI-помощник на базе RAG сначала находит релевантные документы во внешних источниках с помощью, например, векторного поиска, ранжирует найденную информацию, а затем генерирует ответ на запрос пользователя на основе ЭТИХ данных.

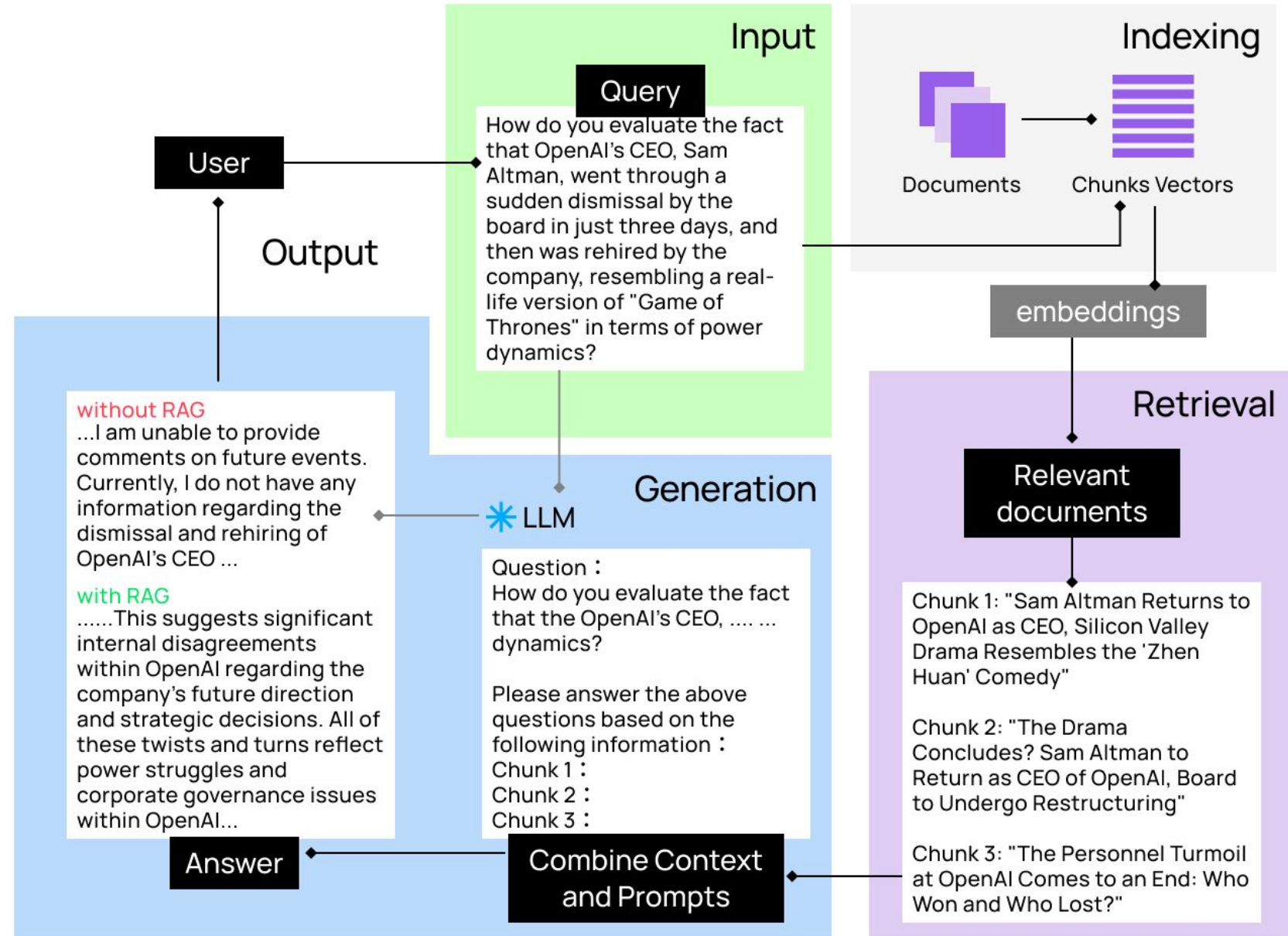
RAG

A typical case of RAG

When answering questions or generating text, it first retrieves relevant information from a large number of documents, and then LLMs generates answers based on this information.

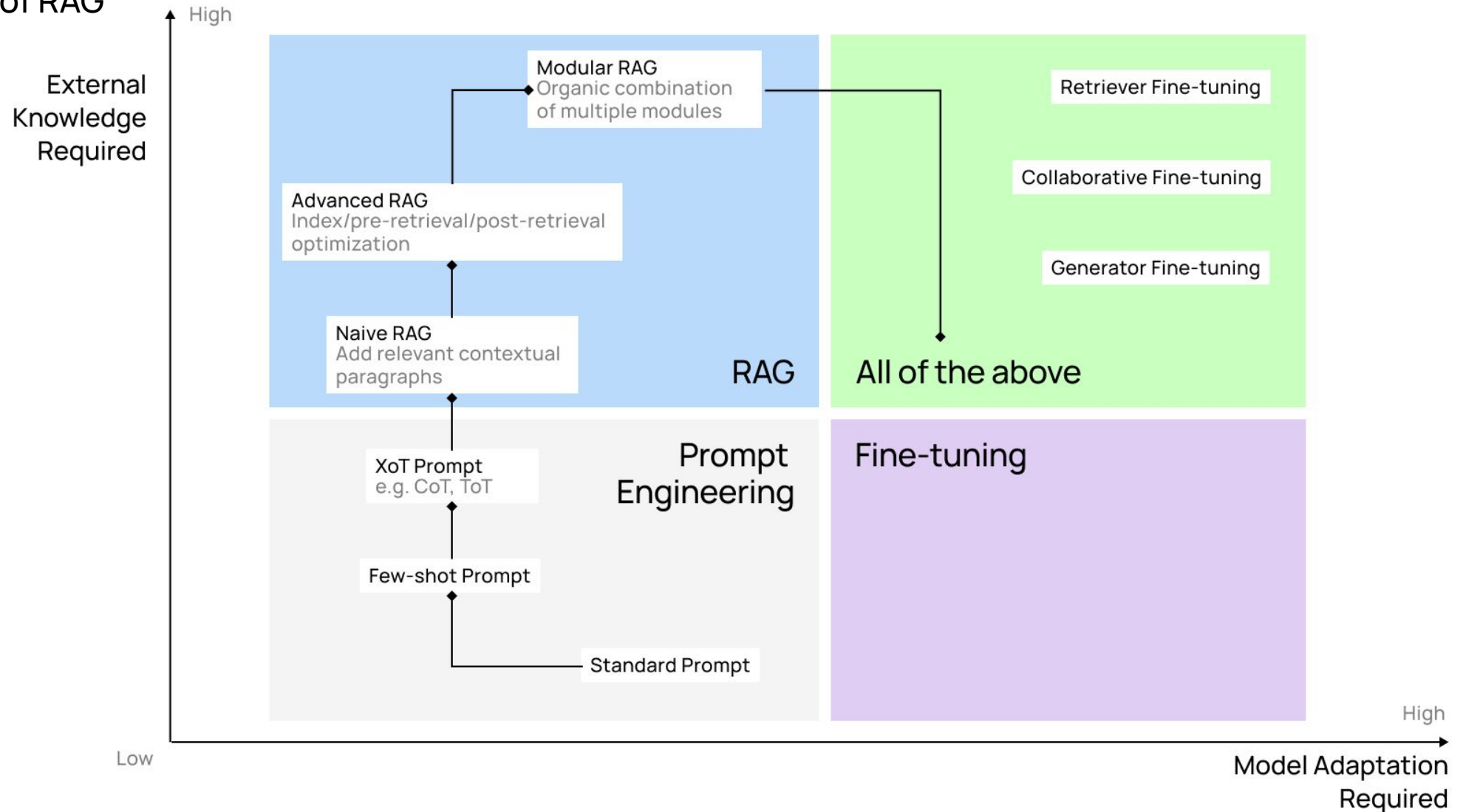
By attaching a external knowledge base, there is no need to retrain the entire large model for each specific task.

The RAG model is especially suitable for knowledge-intensive tasks.



RAG

A typical case of RAG



RAG

Scenarios where RAG is applicable:

- Long-tail distribution of data
- Frequent knowledge updates
- Answers requiring verification and traceability
- Specialized domain knowledge
- Data privacy preservation

Q&A

RETRO (Borgeaud et al 2021)
REALM (Gu et al, 2020)
ATLAS (Izacard et al, 2023)

Fact Checking

RAG (Lewis et al, 2020)
ATLAS (Izacard et al, 2022)
Evi. Generator (Asai et al, 2022a)

Dialog

BlenderBot3 (Shuster et al. 2022)
Internet-augmented generation (Komeili et a., 2022)

Summary

FLARE (Jiang et al, 2023)

Machine Translation

kNN-MT (Khandelwal et al., 2020)
TRIME-MT (Zhong et al., 2022)

Code Generation

DocPrompting (Zhou et al., 2023)
Natural ProverWelleck et al., 2022)

Natural Language Inference

kNN-Prompt (Shi et al., 2022)
NPM (Min et al., 2023)

Sentiment analysis

kNN-Prompt (Shi et al., 2022)
NPM (Min et al., 2023)

Commonsense reasoning

Raco (Yu et al, 2022)

RAG

https://raw.githubusercontent.com/Tongji-KGLLM/RAG-Survey/main/assets/RAG_Slide_ENG.pdf

RAG и ReAct —
уже агент или
нет?

RAG и ReAct –
уже агент или
нет?

Нет.

Summary

LLM — это инструмент,
которым нужно
уметь пользоваться.

